

FILE AUTOMATION SCRIPT

Internspark Python Developer Internship

Name: Sammerpreet kaur

Course: B.Tech in Computer Science and Engineering(CSE)

Date:20 June 2026

Task 1: Python Automation Script

Objective: To develop a python automation script that performs file operations such as renaming, sorting and cleaning the os module, exception handling, logging and user input.

Tools used:

- Python 3.x
- VS code
- OS module
- Logging module
- GitHub

Source code:

```
import os
import shutil
from datetime import datetime

LOG_FILE = "log.txt"

def write_log(message):
    with open(LOG_FILE, "a") as log:
        log.write(f"[{datetime.now()}] {message}\n")

def rename_files(folder_path):
    try:
        files = os.listdir(folder_path)
        count = 1
```

```

        for file in files:
            old_path = os.path.join(folder_path, file)

            if os.path.isfile(old_path):
                extension = os.path.splitext(file)[1]
                new_name = f"File_{count}{extension}"
                new_path = os.path.join(folder_path, new_name)

                os.rename(old_path, new_path)
                write_log(f"Renamed {file} to {new_name}")
                count += 1

        print("Files renamed successfully.")

    except Exception as e:
        print("Error while renaming files:", e)
        write_log(f"Rename Error: {e}")

def sort_files(folder_path):
    try:
        files = os.listdir(folder_path)

        for file in files:
            file_path = os.path.join(folder_path, file)

            if os.path.isfile(file_path):
                extension = os.path.splitext(file)[1][1:].upper()

                if extension == "":
                    extension = "NO_EXTENSION"

                target_folder = os.path.join(
                    folder_path, extension + "_Files"
                )

                os.makedirs(target_folder, exist_ok=True)

                shutil.move(

```

```

        file_path,
        os.path.join(target_folder, file)
    )

    write_log(
        f"Moved {file} to {target_folder}"
    )

    print("Files sorted successfully.")

except Exception as e:
    print("Error while sorting files:", e)
    write_log(f"Sort Error: {e}")

def clean_files(folder_path):
    try:
        unwanted_extensions = [".tmp", ".bak", ".log"]

        files = os.listdir(folder_path)

        for file in files:
            file_path = os.path.join(folder_path, file)

            if os.path.isfile(file_path):
                extension = os.path.splitext(file)[1].lower()

                if extension in unwanted_extensions:
                    os.remove(file_path)

                    write_log(
                        f"Deleted unwanted file {file}"
                    )

        print("Cleaning completed.")

    except Exception as e:
        print("Error while cleaning files:", e)
        write_log(f"Cleaning Error: {e}")

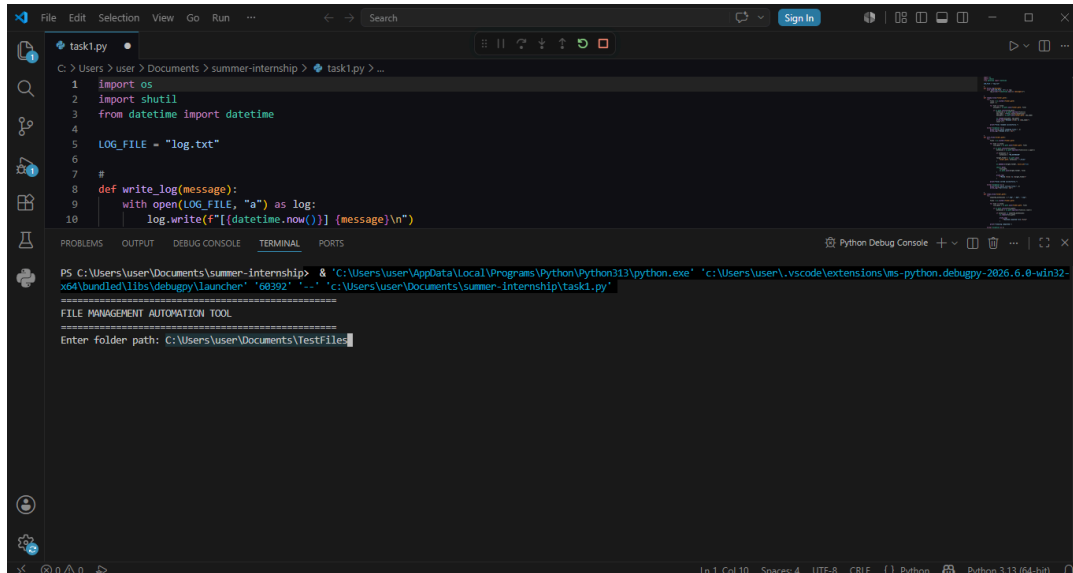
```

```
def main():  
    print("=" * 50)  
    print("FILE MANAGEMENT AUTOMATION TOOL")  
    print("=" * 50)  
  
    folder_path = input("Enter folder path: ")  
  
    if not os.path.exists(folder_path):  
        print("Folder does not exist.")  
        return  
  
    while True:  
        print("\n----- MENU -----")  
        print("1. Rename Files")  
        print("2. Sort Files")  
        print("3. Clean Temporary Files")  
        print("4. Exit")  
  
        choice = input("Enter your choice: ")  
  
        if choice == "1":  
            rename_files(folder_path)  
  
        elif choice == "2":  
            sort_files(folder_path)  
  
        elif choice == "3":  
            clean_files(folder_path)  
  
        elif choice == "4":  
            print("Exiting program...")  
            break  
  
        else:  
            print("Invalid choice. Try again.")
```

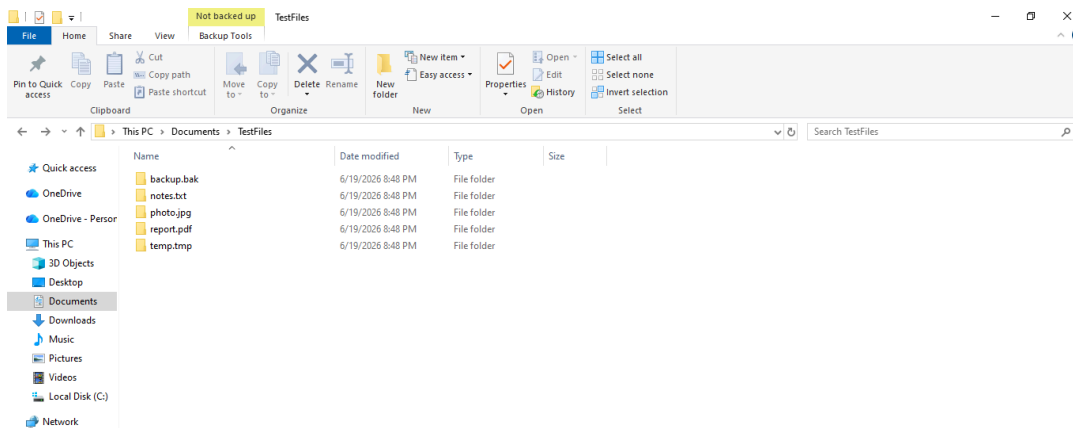
```
if __name__ == "__main__":
    main()
```

Sample Input and Output:

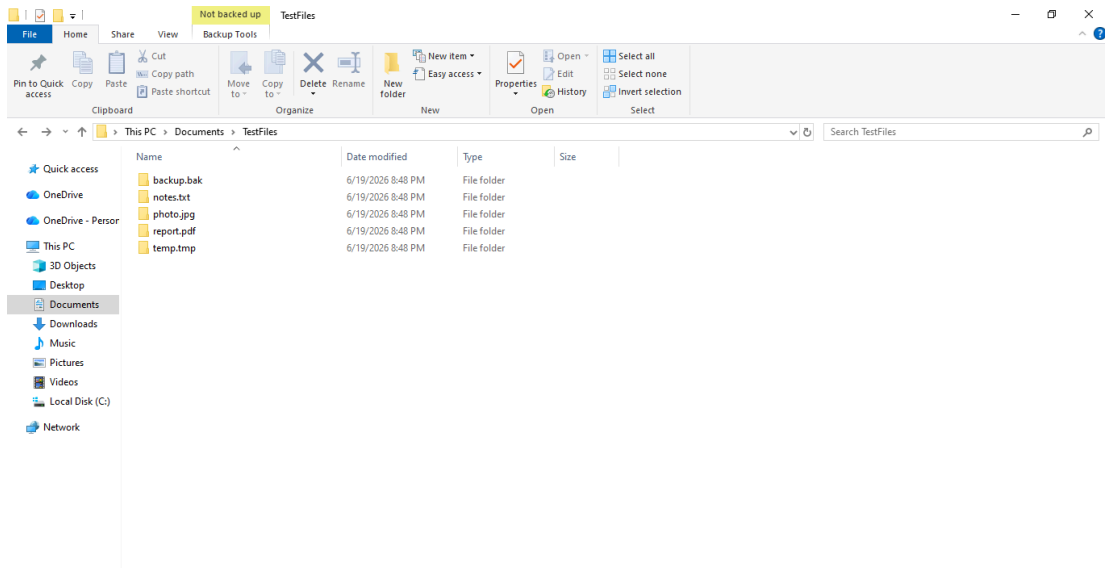
- Screenshot 1- User input



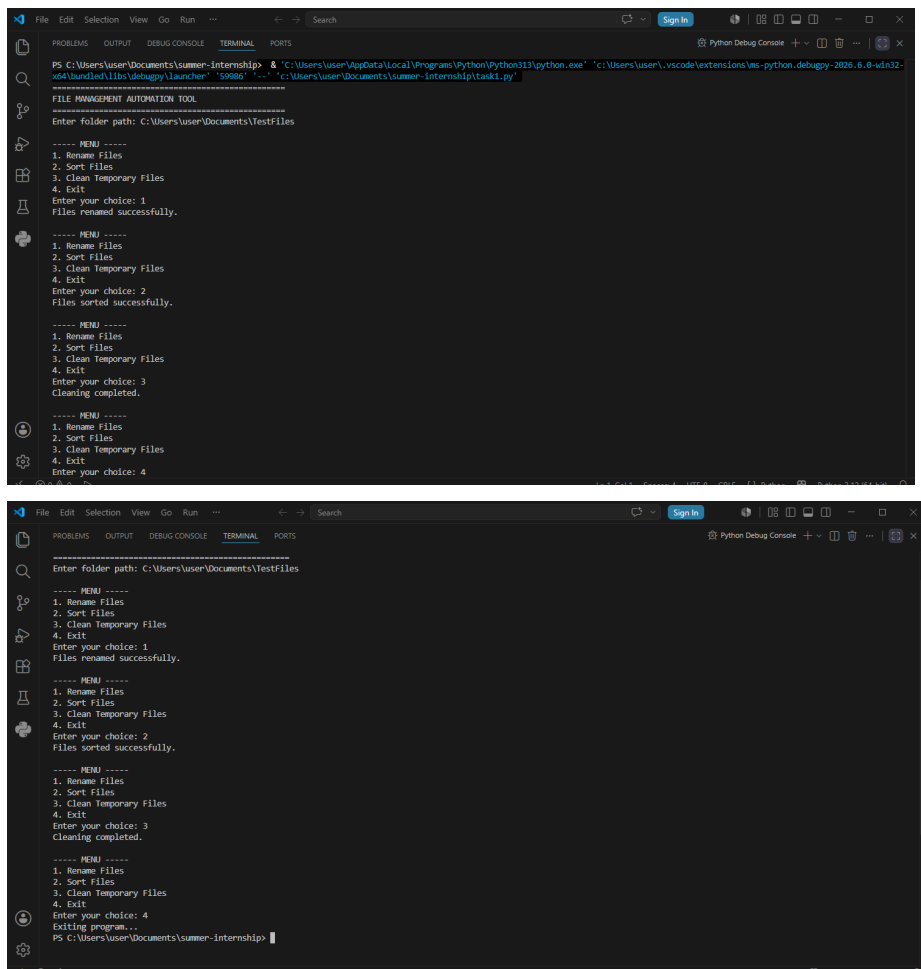
- Screenshot 2- Folder before operation



- Screenshot 3- Folder after operation

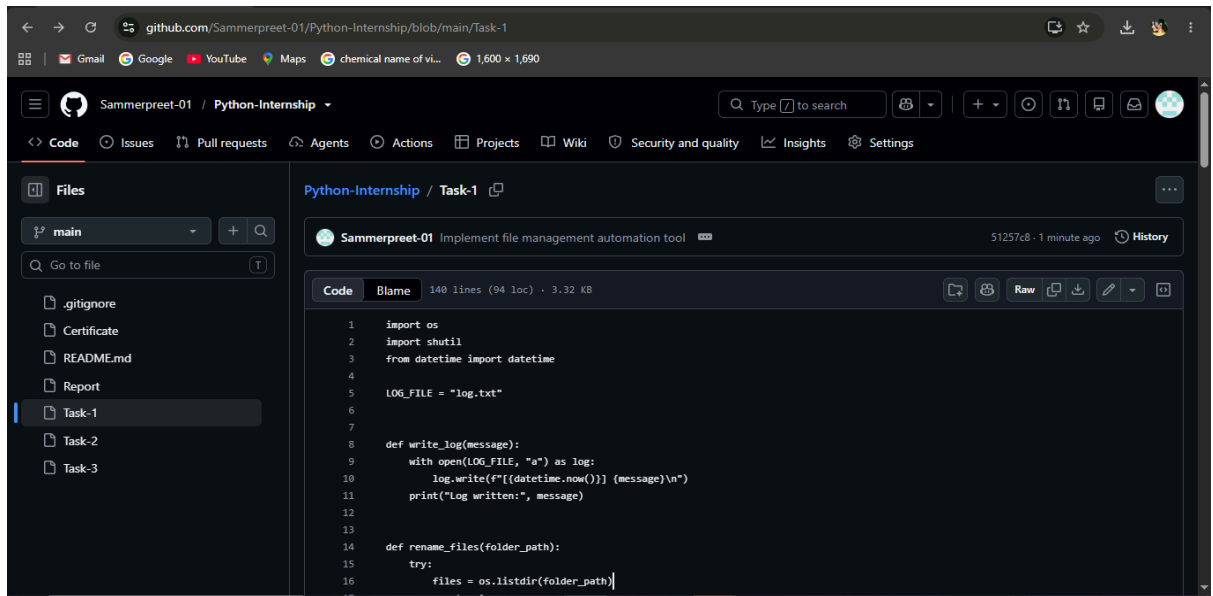


- Screenshot 4- Terminal output



GitHub Source Code

- Repository Name: Python-Internship
- Repository Link: <https://github.com/Sammerpreet-01/Python-Internship.git>
- Repository Screenshot



:

Features Implemented:

- User input support
- File renaming
- File sorting
- Logging of operations
- OS module usage
- Exception handling

Conclusion: The automation script successfully performs file management tasks including sorting and renaming files. The project demonstrates the use of Python's OS module, exception handling, user interaction and logging.